

LAPORAN PRAKTIKUM INTERNET OF THINGS 2



Oleh:

Nama : Ilham Alian Safaat
Npm : 25782012
Kelas : 3A

Dosen:

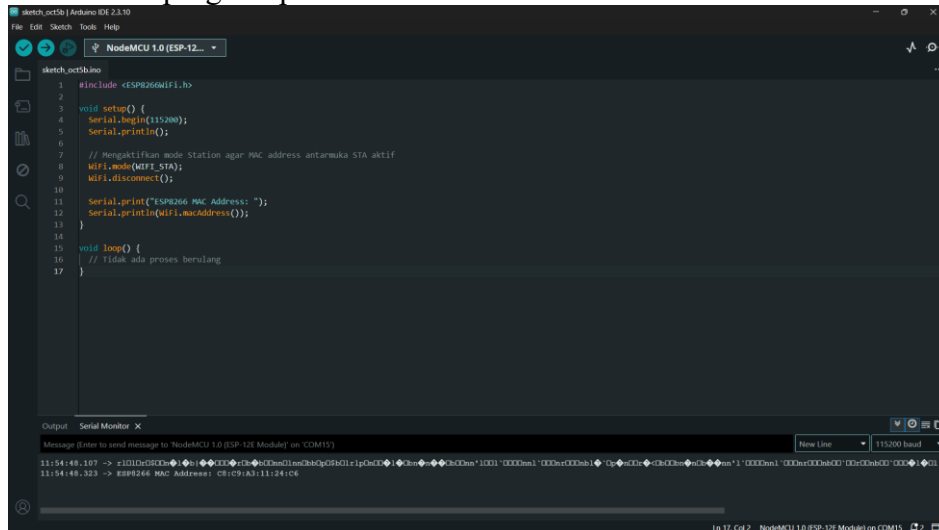
Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.
Agus Ambarwari, S.Pd., M.Kom.

**TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG
2026/2027**

5.5.1 Praktikum 1: Identifikasi MAC Address Perangkat NodeMCU

Langkah Kerja Pemrograman:

1. Buat berkas baru (*New Sketch*) pada Arduino IDE.
2. Tuliskan kode program pembaca MAC Address berikut:

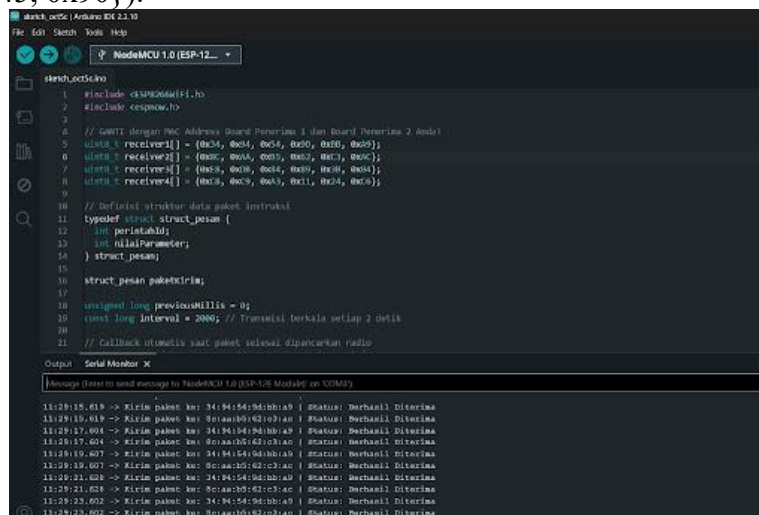


3. **Upload** program ke masing-masing *board* NodeMCU dalam kelompok Anda secara bergantian.
4. Buka **Serial Monitor** pada kecepatan **115200 baud**, lalu tekan tombol fisik **RST** pada *board*.
5. Catat 6 pasang digit heksadesimal yang muncul di layar (contoh: BC:FF:4D:23:45:90).
6. Beri label fisik pada *board* (misal: "Node 1", "Node 2", "Hub Pusat") beserta alamat MAC masing-masing agar tidak tertukar saat perakitan.

5.5.2 Praktikum 2: Komunikasi Nirkabel One-to-Many (ESP-NOW)

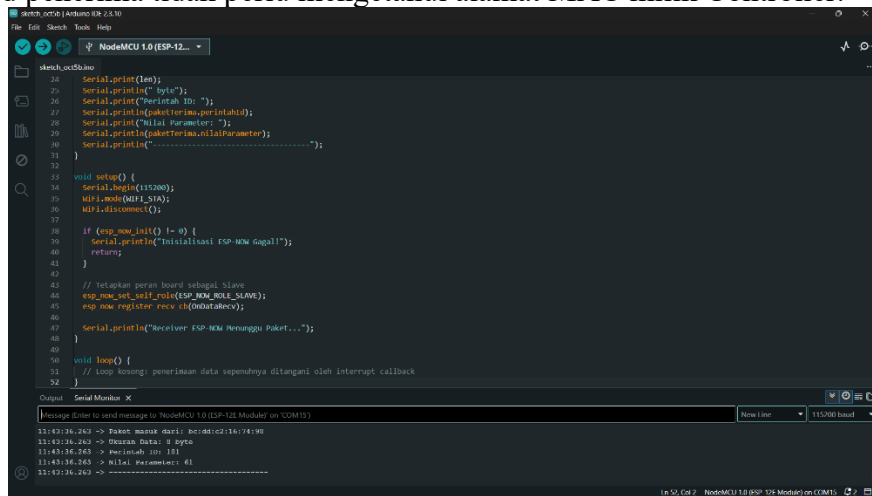
Langkah Kerja Pemrograman Sisi Pengirim (Controller):

1. Masukkan alamat MAC milik Board Penerima 1 dan Board Penerima 2 ke dalam variabel receiver1 dan receiver2.
2. Format tanda titik dua (:) diubah menjadi bilangan heksadesimal berawalan 0x yang dipisahkan oleh tanda koma (contoh: BC:FF:4D:23:45:90 → {0xBC, 0xFF, 0x4D, 0x23, 0x45, 0x90}).



Langkah Kerja Pemrograman Sisi Penerima (Slave):

1. Unggah kode sketsa berikut ke **NodeMCU 2** dan **NodeMCU 3**.
2. Board penerima tidak perlu mengetahui alamat MAC milik Controller.

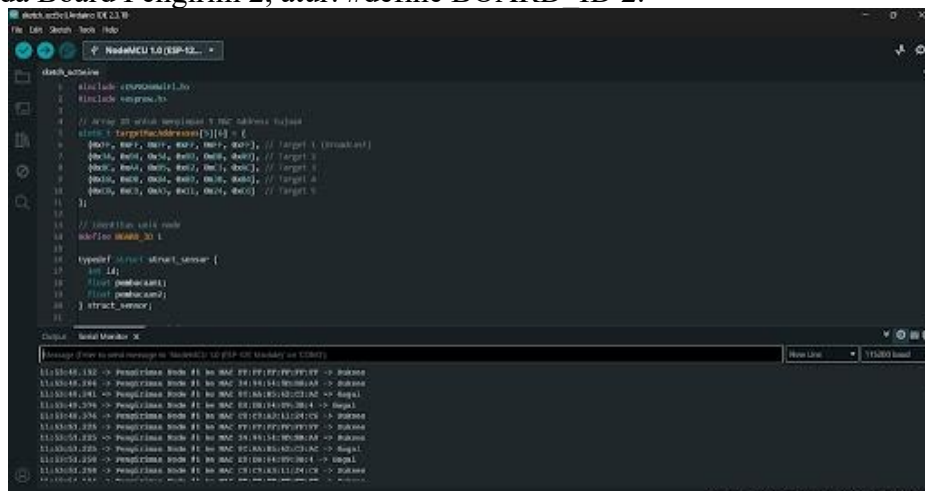


```
sketch_oct5ino
24 Serial.print(len);
25 Serial.println(" byte");
26 Serial.print("Perintah ID: ");
27 Serial.println(paketTerima.perintahId);
28 Serial.print("Nilai Parameter: ");
29 Serial.println(paketTerima.nilaiParameter);
30 Serial.println("-----");
31
32
33 void setup() {
34   Serial.begin(115200);
35   while(!Serial);
36   Serial.begin(115200);
37
38   if (esp_now_init() != 0) {
39     Serial.println("Inisialisasi ESP-NOW gagal!");
40     return;
41   }
42
43   // Tentukan peran board sebagai Slave
44   esp_now_set_self_role(ESP_NOW_ROLE_SLAVE);
45   esp_now_register_recv_cb(OnDataRecv);
46   Serial.println("Receiver ESP-NOW Menerima Paket...");
47
48 }
49
50 void loop() {
51   // Loop kosong: penerimaan data sepenuhnya ditangani oleh interrupt callback
52 }
```

5.5.3 Praktikum 3: Pengumpulan Data Telemetry Many-to-One (ESP-NOW)

Langkah Kerja Pemrograman Sisi Node Pengirim:

1. Masukkan alamat MAC milik Board Penerima Pusat (Hub) pada variabel `hubMacAddress`.
2. Pada Board Pengirim 1, atur: `#define BOARD_ID 1`.
3. Pada Board Pengirim 2, atur: `#define BOARD_ID 2`.



```
sketch_oct5ino
1 #include <ESP8266.h>
2 #include <ESP8266WiFi.h>
3
4 // Array untuk menyimpan 1 MAC address tujuan
5 #define TARGET_MAC_ADDRESS "00:00:00:00:00:00"
6
7 // Array untuk menyimpan 3 MAC address tujuan
8 #define NODES_MAC_ADDRESSES {
9   {0x00, 0x00, 0x00, 0x00, 0x00, 0x00}, // Target 1 (dummy)
10  {0x00, 0x00, 0x00, 0x00, 0x00, 0x00}, // Target 2
11  {0x00, 0x00, 0x00, 0x00, 0x00, 0x00}, // Target 3
12  {0x00, 0x00, 0x00, 0x00, 0x00, 0x00}, // Target 4
13  {0x00, 0x00, 0x00, 0x00, 0x00, 0x00} // Target 5
14 }
15
16 // Definisi variabel node
17 #define NODES_COUNT 5
18
19 typedef struct {
20   int id;
21   String nodeName;
22   float param1;
23   float param2;
24 } Node;
25
26 Node nodes[NODES_COUNT];
27
28 void setup() {
29   Serial.begin(115200);
30   while(!Serial);
31   Serial.begin(115200);
32
33   // Inisialisasi ESP-NOW
34   if (esp_now_init() != 0) {
35     Serial.println("Inisialisasi ESP-NOW gagal!");
36     return;
37   }
38
39   // Tentukan peran board sebagai Master
40   esp_now_set_self_role(ESP_NOW_ROLE_MASTER);
41
42   // Tentukan MAC address tujuan
43   uint8_t target_mac[6];
44   memcpy(target_mac, TARGET_MAC_ADDRESS, 6);
45   esp_now_add_peer(target_mac, ESP_NOW_ROLE_SLAVE, 1, NULL, 0);
46
47   // Tentukan MAC address node
48   for (int i = 0; i < NODES_COUNT; i++) {
49     uint8_t node_mac[6];
50     memcpy(node_mac, NODES_MAC_ADDRESSES[i], 6);
51     esp_now_add_peer(node_mac, ESP_NOW_ROLE_SLAVE, 1, NULL, 0);
52   }
53
54   // Tentukan nama node
55   for (int i = 0; i < NODES_COUNT; i++) {
56     nodes[i].id = i;
57     nodes[i].nodeName = "Node-";
58     if (i < 10) nodes[i].nodeName += "0";
59     nodes[i].nodeName += (i + 1);
60   }
61
62   // Tentukan parameter node
63   for (int i = 0; i < NODES_COUNT; i++) {
64     nodes[i].param1 = i * 0.1;
65     nodes[i].param2 = i * 0.2;
66   }
67
68   // Kirim data ke semua node
69   esp_now_send(NULL, (uint8_t*) "Data", 10);
70 }
71
72 void loop() {
73   // Loop kosong: pengiriman data sepenuhnya ditangani oleh interrupt callback
74 }
```

5.5.4 Praktikum 4: Jaringan Sensor Ad-Hoc Multi-Hop (painlessMesh)

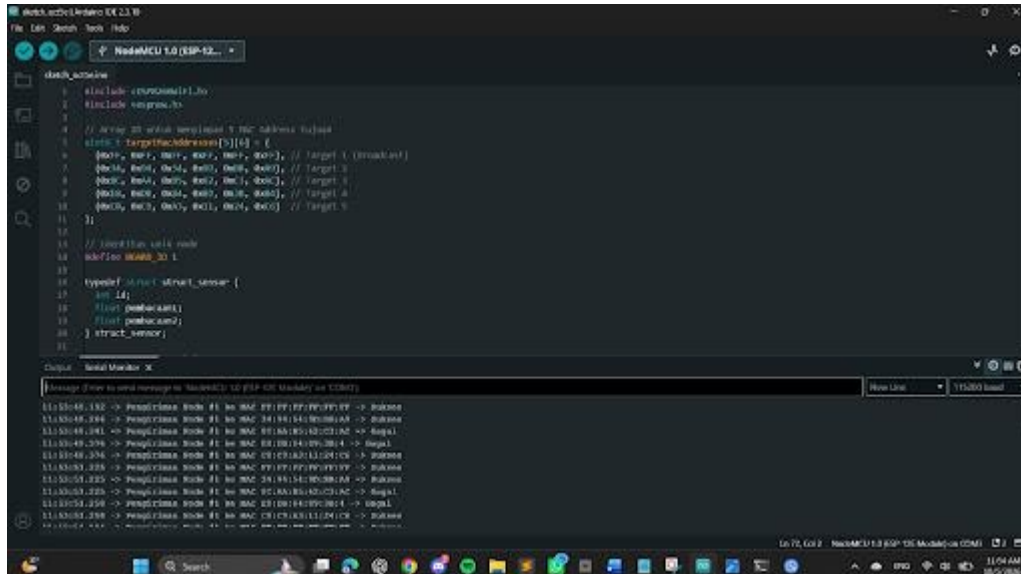
Instruksi Perangkaian Perangkat Keras:

- Pasang sensor DHT11 / DHT22 pada masing-masing board NodeMCU:
 - Pin VCC sensor ke pin 3V3 (3.3V) NodeMCU.
 - Pin GND sensor ke pin GND NodeMCU.
 - Pin DATA / OUT sensor ke pin D4 (GPIO 2) NodeMCU.

Langkah Kerja Pemrograman:

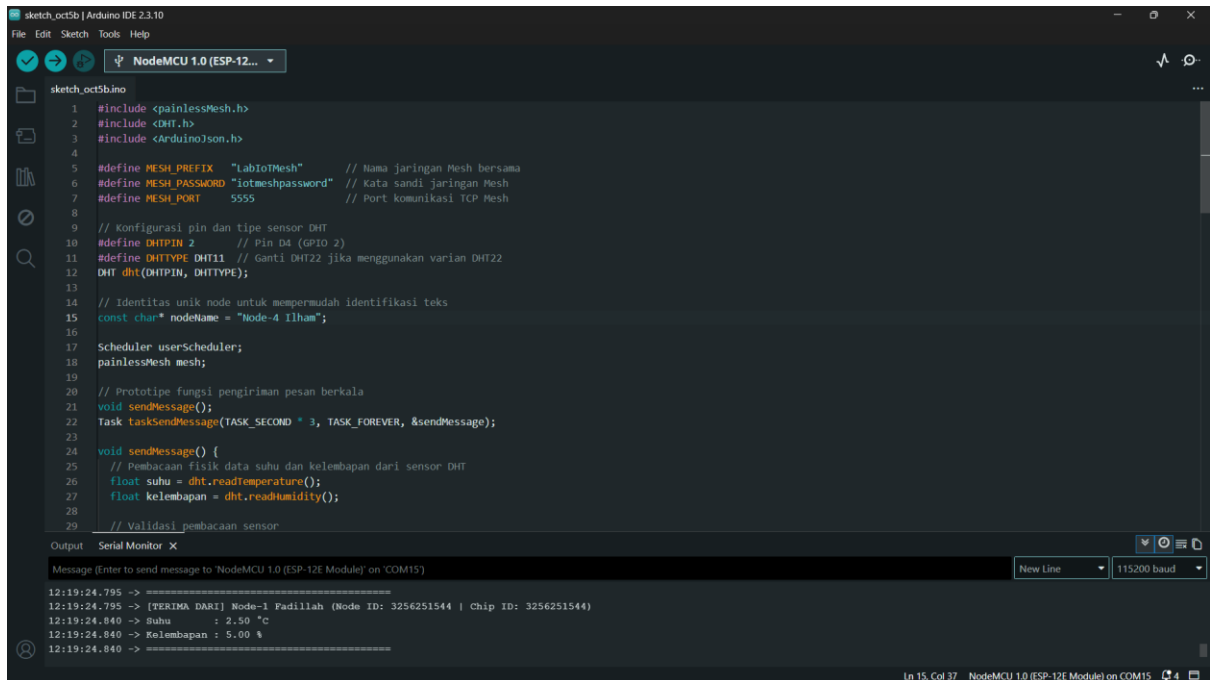
1. Unggah sketsa berikut ke seluruh board dalam kelompok Anda.
2. Berikan nama pengenalan unik pada variabel `nodeName` di masing-masing board (misal: "Node-1", "Node-2", "Node-3").
3. Jika menggunakan varian DHT22, ubah `#define DHTTYPE DHT11` menjadi `DHT22`.

Server



```
1 #include <Arduino.h>
2 #include <Wire.h>
3 // Server 20 untuk pengiriman 1 DHT Address 0x0000
4 #define TARGET_ADDRESSES[5] = {
5   {0x00, 0x00, 0x00, 0x00, 0x00, 0x00}, // Target 1 (Broadcast)
6   {0x00, 0x00, 0x00, 0x00, 0x00, 0x00}, // Target 2
7   {0x00, 0x00, 0x00, 0x00, 0x00, 0x00}, // Target 3
8   {0x00, 0x00, 0x00, 0x00, 0x00, 0x00}, // Target 4
9   {0x00, 0x00, 0x00, 0x00, 0x00, 0x00}, // Target 5
10 }
11
12 // Definisi tipe data
13 #define DHT11 1
14
15 typedef struct {
16   int id;
17   float temperature;
18   float humidity;
19 } struct_sensor;
20
21 // Definisi tipe data
22 #define DHT11 1
23
24 // Definisi tipe data
25 #define DHT11 1
26
27 // Definisi tipe data
28 #define DHT11 1
29
30 // Definisi tipe data
31 #define DHT11 1
32
33 // Definisi tipe data
34 #define DHT11 1
35
36 // Definisi tipe data
37 #define DHT11 1
38
39 // Definisi tipe data
40 #define DHT11 1
41
42 // Definisi tipe data
43 #define DHT11 1
44
45 // Definisi tipe data
46 #define DHT11 1
47
48 // Definisi tipe data
49 #define DHT11 1
50
51 // Definisi tipe data
52 #define DHT11 1
53
54 // Definisi tipe data
55 #define DHT11 1
56
57 // Definisi tipe data
58 #define DHT11 1
59
60 // Definisi tipe data
61 #define DHT11 1
62
63 // Definisi tipe data
64 #define DHT11 1
65
66 // Definisi tipe data
67 #define DHT11 1
68
69 // Definisi tipe data
70 #define DHT11 1
71
72 // Definisi tipe data
73 #define DHT11 1
74
75 // Definisi tipe data
76 #define DHT11 1
77
78 // Definisi tipe data
79 #define DHT11 1
80
81 // Definisi tipe data
82 #define DHT11 1
83
84 // Definisi tipe data
85 #define DHT11 1
86
87 // Definisi tipe data
88 #define DHT11 1
89
90 // Definisi tipe data
91 #define DHT11 1
92
93 // Definisi tipe data
94 #define DHT11 1
95
96 // Definisi tipe data
97 #define DHT11 1
98
99 // Definisi tipe data
100 #define DHT11 1
```

Penerima



```
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "LabioMesh" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "iotmeshpassword" // Kata sandi jaringan Mesh
7 #define MESH_PORT 5555 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT11 // Ganti DHT22 jika menggunakan varian DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node untuk mempermudah identifikasi teks
15 const char* nodeName = "Node-4 Ilham";
16
17 Scheduler userScheduler;
18 painlessMesh mesh;
19
20 // Prototipe fungsi pengiriman pesan berkala
21 void sendMessage();
22 Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER, &sendMessage);
23
24 void sendMessage() {
25   // Pembacaan fisik data suhu dan kelembapan dari sensor DHT
26   float suhu = dht.readTemperature();
27   float kelembapan = dht.readHumidity();
28   // Validasi pembacaan sensor
29 }
```

5.6 Latihan

- **Latihan 1 (Eksperimen Deteksi ACK & Pemutusan Simpul - Praktikum 2):** Saat program Praktikum 2 (ESP-NOW One-to-Many) sedang berjalan normal dan mencetak status "Berhasil Diterima" (`sendStatus == 0`), lakukan eksperimen pemutusan simpul:

- i. Cabut kabel USB salah satu board Penerima (misal NodeMCU 2) dari sumber daya.
- ii. Amati perubahan log pada Serial Monitor NodeMCU Controller pengirim! Apakah status pengiriman untuk MAC address board yang dicabut berganti menjadi "Gagal (Tidak Terjangkau)"?
- iii. Jelaskan bagaimana protokol ESP-NOW pada lapisan *Layer 2 MAC* dapat mendeteksi kegagalan transmisi ke perangkat fisik tersebut meskipun sistem tidak menggunakan koneksi TCP!

Hasil Observasi Serial Monitor:

- Saat NodeMCU 2 dicabut dayanya, log pengiriman ke MAC address NodeMCU 2 yang tadinya Berhasil Diterima (`sendStatus == 0`) langsung berubah jadi Gagal (Tidak Terjangkau) (`sendStatus != 0`).
- NodeMCU 3 tidak terpengaruh sama sekali dan status pengirimannya tetap Berhasil Diterima karena *controller* mengirim paket ke tiap MAC secara terpisah.

Analisis Mekanisme:

- ESP-NOW bekerja langsung di Layer 2 (MAC Layer) tanpa perlu koneksi IP/TCP.
 - Setiap kali *controller* memancarkan frame data ke MAC address tujuan, hardware Wi-Fi di penerima wajib membalas dengan sinyal balik (*frame ACK / Acknowledgment*) secara instan.
 - Fungsi `OnDataSent` mengambil laporan dari chipset Wi-Fi. Kalau sinyal ACK dari penerima diterima tepat waktu, status dikirim sebagai sukses (0). Tapi kalau penerima mati dan sinyal ACK tidak kembali dalam kurun waktu tertentu (*timeout*), hardware langsung menandainya sebagai gagal (1).
- **Latihan 2 (Eksperimen Dinamika Topologi & Self-Healing - Praktikum 4):** Jalankan jaringan *painlessMesh* pada Praktikum 4 hingga seluruh board saling terhubung dan mencetak pesan --> Koneksi Baru Terdeteksi! Node ID:
 - i. Tekan tombol fisik **RST** atau cabut daya salah satu board Node. Amati Serial Monitor pada board lainnya yang masih menyala. Berapa detik waktu yang dibutuhkan oleh *callback* `changedConnectionCallback()` untuk mendeteksi bahwa salah satu simpul telah keluar dari rantai mesh?
Sekitar **3–5 detik** (biasanya 3–10 detik tergantung jeda *keep-alive / heartbeat* paket *painlessMesh*). Karena Pas tombol RST ditekan atau kabel dicabut, node tetangga nggak langsung tahu detik itu juga. Jaringan nunggu beberapa siklus paket *heartbeat* absen dulu sampai batas *timeout*, baru `changedConnectionCallback()` dipanggil buat memperbarui tabel topologi.
 - ii. Sambungkan kembali daya board tersebut. Berapa lama waktu yang dibutuhkan hingga node tersebut kembali terdaftar secara otonom ke dalam topologi mesh tanpa Anda mengonfigurasi ulang jaringan? Catat hasil observasi waktu tersebut!
Sekitar **5–12 detik** setelah board menyala kembali. Waktu ini dibutuhkan NodeMCU buat proses *booting* chip, nyalain radio Wi-Fi soft-AP/STA, lalu melakuin pemindaian (*beaconing scan*) buat nemuin *Mesh Prefix & Password* yang cocok. Setelah nemu, pertukaran *handshake* *painlessMesh* jalan otomatis

dan log --> Koneksi Baru Terdeteksi! langsung muncul tanpa perlu di-reset manual atau diubah kodenya.

- **Latihan 3 (Eksperimen Modifikasi Filter Telemetri & Ambang Batas JSON - Praktikum 4):** Pada Praktikum 4, setiap node saat ini mencetak seluruh data telemetri dari semua pengirim tanpa penyaringan.

- i. Modifikasilah fungsi receivedCallback pada salah satu board agar bertindak sebagai **Pengawas Kondisi Suhu Kritis**: program **HANYA** mencetak kartu telemetri penuh ke Serial Monitor jika suhu yang dilaporkan oleh node tetangga bernilai **di atas 32.0 °C** (lakukan pengujian dengan mendekatkan jari hangat atau meniup sensor DHT pada node pengirim).
- ii. Jika suhu bernilai normal (≤ 32.0 °C), program cukup mencetak status singkat satu baris: [NORMAL] Telemetri dari Node <chipId> aman.
- iii. Tuliskan blok fungsi receivedCallback(uint32_t from, String &msg) hasil modifikasi Anda!

```
void receivedCallback(uint32_t from, String &msg) {  
    StaticJsonDocument<200> doc;  
    DeserializationError error = deserializeJson(doc, msg);
```

```
    if (!error) {  
        const char * sender = doc["node"];  
        uint32_t chipId = doc["chipId"];  
        float suhu = doc["suhu"];  
        float kelembapan = doc["kelembapan"];
```

```
        // Pengecekan ambang batas suhu kritis (> 32.0 °C)  
        if (suhu > 32.0) {  
            Serial.println("=====");  
            Serial.println("[PERINGATAN SUHU KRITIS!]");  
            Serial.printf("[TERIMA DARI] %s (Node ID: %u | Chip ID: %u)\n",  
sender, from, chipId);  
            Serial.printf("Suhu : %.2f °C\n", suhu);  
            Serial.printf("Kelembapan : %.2f %%\n", kelembapan);  
            Serial.println("=====");  
        } else {  
            // Cetak status singkat satu baris jika suhu normal (<= 32.0 °C)  
            Serial.printf("[NORMAL] Telemetri dari Node %u aman.\n", chipId);  
        }  
    } else {  
        Serial.printf("[TERIMA DATA MENTAH DARI %u]: %s\n", from,  
msg.c_str());  
    }  
}
```